Active Job Basics

This guide provides you with all you need to get started in creating, enqueueing and executing background jobs.

1 Introduction

Active Job is a framework for declaring jobs and making them run on a variety of queueing backends. These jobs can be everything from regularly scheduled clean-ups, to billing charges, to mailings. Anything that can be chopped up into small units of work and run in parallel, really.

2 The Purpose of Active Job

The main point is to ensure that all Rails apps will have a job infrastructure in place, even if it's in the form of an "immediate runner". We can then have framework features and other gems build on top of that, without having to worry about API differences between various job runners such as Delayed Job and Resque. Picking your queuing backend becomes more of an operational concern, then. And you'll be able to switch between them without having to rewrite your jobs.

3 Creating a Job

This section will provide a step-by-step guide to creating a job and enqueuing it.

3.1 Create the Job

Active Job provides a Rails generator to create jobs. The following will create a job in `app/jobs` (with an attached test case under `test/jobs`):

```
$ bin/rails generate job guests_cleanup
invoke  test_unit
create    test/jobs/guests_cleanup_job_test.rb
create  app/jobs/guests_cleanup_job.rb
```

You can also create a job that will run on a specific queue:

```
$ bin/rails generate job guests_cleanup --queue urgent
```

If you don't want to use a generator, you could create your own file inside of `app/jobs`, just make sure that it inherits from `ActiveJob::Base`.

Here's what a job looks like:

```
class GuestsCleanupJob < ActiveJob::Base
  queue_as :default

  def perform(*args)
    # Do something later
  end
end
```

3.2 Enqueue the Job

Enqueue a job like so: